PYTHON PROGRAMMING

ASSIGNMENT 4 – CO4

NAME: M.Sharmila

REG NO: 192572139

GITHUB LINK:
https://github.com/sharmilamohan12/python/tree/main/CO%20TOOLS

ROBUST DATA PROCESSING WITH FILES, EXCEPTIONS, MODULES AND PACKAGES

Course: Python Programming (CSA08)

TASK 1 – FILE-PROCESSING DESIGN REVIEW

S.No	Design Deficiency	Problem
1	Hard-coded file path	Difficult to change the file location.
2	File not closed	May waste system resources.
3	Reads entire file	Consumes more memory.
4	Bare except	Hides all errors.
5	Uses int()	Cannot handle decimal amounts.
6	Repeated +	Makes report creation inefficient.
7	No rejected-line record	Invalid records cannot be checked later.

IMPLEMENTATION:

CO2 PYTHON.py - C:/Users/M. Sharmila/OneDrive/Documents/CO2 PYTHON.py (3.14.2)

File Edit Format Run Options Window Help

```
def calculate_bill(filename):
    total = 0.0
    report = []
    rejected = []

    try:
        with open(filename, "r") as f:
            for line in f:
                try:
                    parts = line.strip().split(",")

                    if len(parts) != 3:
                        raise ValueError

                    name = parts[1]
                    amount = float(parts[2])

                    total += amount
                    report.append(name + " " + parts[2])

                except (ValueError, IndexError):
                    rejected.append(line.strip())

        print("MESS BILL REPORT")
        for item in report:
            print(item)

        print("Total:", total)
        print("Rejected Lines:", rejected)

    except FileNotFoundError:
        print("File not found!")


calculate_bill("C:\\Users\\Office\\Desktop\\mess_oct.txt")
```

OUTPUT:

```
MESS BILL REPORT
Arun 1250.50
Bala 1500.00
Divya 1800.75
Kumar 950.00

Total: 5501.25

Rejected Lines: ['Invalid data']
```

TASK 2 – FROM REQUIREMENT TO FILE-PROCESSING DESIGN

Task 2.1 – Model the DataRegisterNo,Date,Status

101,01-11-2026,P

102,01-11-2026,A

103,01-11-2026,P

Task 2.2 – Design the File Operations (2 Marks)

Operation	Usage
r	Read attendance files.
w	Create attendance report.
a	Add damaged records to error log.
UTF-8	Supports text encoding.
with open()	Automatically closes files.
write()	Writes reports.
Iteration	Reads files line by line.

Task 2.3 – Design the Processing

```
def attendance_report(folder):
    attendance = {}
    errors = []

    files = os.listdir(folder)

    for filename in files:
        if filename.endswith(".txt"):
            path = os.path.join(folder, filename)

            with open(path, "r", encoding="utf-8") as f:
                for line_no, line in enumerate(f, 1):
                    parts = line.strip().split(",")

                    try:
                        if len(parts) != 3:
                            raise ValueError

                        reg, date, status = parts

                        if status not in ["P", "A"]:
                            raise ValueError

                        if reg not in attendance:
                            attendance[reg] = [0, 0]

                        if status == "P":
                            attendance[reg][0] += 1
                        else:
                            attendance[reg][1] += 1

                    except ValueError:
                        errors.append(
                            f"{filename}, Line {line_no}: {line.strip()}"
                        )

    with open("attendance_report.txt", "w", encoding="utf-8") as f:
        for reg, count in attendance.items():
            total = count[0] + count[1]
            percentage = (count[0] / total) * 100

            f.write(f"{reg}: {percentage:.2f}%\n")

            if percentage < 75:
                f.write("Below 75% attendance\n")

    with open("error_log.txt", "a", encoding="utf-8") as f:
```

error_log.txt

attendance1.txt, Line 3: 103,01-11-2026,X

TASK 3 – DESIGN THE ERROR HANDLING

Task 3.1 – Identify the Failures (3 Marks)

Failure	**Exception**
Missing folder/file	FileNotFoundError
Permission denied	PermissionError

Failure	Exception
Wrong encoding	UnicodeDecodeError
Missing fields	InvalidRecord
Invalid status	InvalidRecord
Missing list element	IndexError
Invalid number	ValueError
Empty file	No exception (handle separately)

IMPLEMENTATION:

```
import os
import tempfile

class InvalidRecord(Exception):
    pass


def parse_line(line):
    parts = line.strip().split(",")

    if len(parts) != 3:
        raise InvalidRecord(
            f"expected 3 fields, found {len(parts)}"
        )

    if parts[2] not in ("P", "A"):
        raise InvalidRecord(f"invalid status '{parts[2]}'")

    return tuple(parts)


def summarise(path):
    present, absent, errors = 0, 0, []

    try:
        with open(path, "r", encoding="utf-8") as f:
            for line_no, line in enumerate(f, 1):
                try:
                    reg, date, status = parse_line(line)

                    if status == "P":
                        present += 1
                    else:
                        absent += 1

                except InvalidRecord as e:
                    errors.append((line_no, str(e)))

    except (FileNotFoundError, PermissionError,
            UnicodeDecodeError, OSError) as e:
        raise RuntimeError("File processing failed") from e

    else:
        print("File processed successfully")

    finally:
```

OUTPUT:

```
>>
======== RESTART: C:/Users/M. Sharmila/OneDrive/Documents/CO2 PYTHON.py ========
File processed successfully
File processing completed
(1, 1, [(3, 'expected 3 fields, found 2'), (4, "invalid status 'L'")])
>>
```

Observation:

- Errors are silently ignored.
- Typing mistakes are hidden.
- Debugging becomes difficult.
- Incorrect results may be produced.

Reason: pass ignores the exception without displaying an error message.

TASK 4 – FILE AND EXCEPTION FORENSICS

Task 4.1 – Locate the Defects (2 Marks)

1. **Write mode (w):** Deletes previous records every day.
2. **Missing newline (\n):** All book names join together.
3. **No finally:** File may remain open if writing fails.
4. **No error log:** Errors are displayed or ignored without being saved.

Python Code:

```
def save_returns(books):
    log = open("returns.log", "w")
    for b in books:
        log.write(b)
    log.close()


def process(day_file):
    try:
        books = open(day_file).read().split("\n")
        save_returns(books)
    except FileNotFoundError:
        print("missing")


# Three days of testing
for day, books in enumerate([
    ["Book1", "Book2"],
    ["Book3", "Book4"],
    ["Book5", "Book6"]
], 1):
    with open(f"day{day}.txt", "w") as f:
        f.write("\n".join(books))

    process(f"day{day}.txt")

with open("returns.log", "r") as f:
    print(f.read())
```

OUTPUT:

```
--------
Book5Book6
|
```

Task 4.3 – Repair the Program

```
def save_returns(books):
    with open("returns.log", "a") as log:
        for b in books:
            log.write(b + "\n")


def process(day_file):
    try:
        with open(day_file, "r") as f:
            books = f.read().splitlines()

        save_returns(books)

    except (FileNotFoundError, PermissionError, OSError) as e:
        with open("error.log", "a") as log:
            log.write(str(e) + "\n")


for day in range(1, 4):
    process(f"day{day}.txt")
```

Complexity Analysis:

- Both methods have the same time complexity: O(n).
- Both have the same space complexity: O(n) for reading the file contents.
- with automatically closes files.
- try/finally also closes files if properly implemented.

- If an error occurs, neither correctly implemented version leaves its managed file open.

TASK 5 – THE SAME UTILITY ORGANISED FOUR WAYS

Task 5.1 – Four Organisations

(a) One Script

All functions are written in a single Python file

IMPLEMENTATION:

```
'''
def read_data():
    return {"101": [20, 2], "102": [15, 7]}

def percentage(present, absent):
    return present / (present + absent) * 100

def write_report(data):
    for reg, count in data.items():
        p = percentage(count[0], count[1])
        print(reg, p)

if __name__ == "__main__":
    data = read_data()
    write_report(data)
```

OUTPUT:

```
101 90.9090909090909
102 68.18181818181817
'
```

import attendance

print(attendance.percentage(20, 2))

(c) Three Modules in One Folder

Folder structure:

attendance/

reader.py

stats.py

report.py

main.py

reader.py

def read_data(): return {"101": [20, 2]}

stats.py

def percentage(p, a): return p / (p + a) * 100

report.py

def write_report(data): from stats import percentage for reg, count in data.items(): print(reg, percentage(count[0], count[1]))

main.py

from reader import read_datafrom report import write_reportwrite_report(read_data())

(d) Package Organisation

Folder structure:

attendance/

__init__.py

reader.py

stats.py

TASK 6 – DESIGN CHALLENGE

College Fest Registration Desk – 13 Marks

Stage 1 – Interpret the Problem (2 Marks)

Inputs:

- Five department registration text files.
- Register number, name, department, e-mail and events.

Outputs:

1. master_list.txt – Valid registrations.
2. Event-wise files – Participant lists.
3. summary.txt – Total registrations and participants.
4. error_log.txt – Invalid and damaged records.

Rules:

- Remove duplicate register numbers.
- Validate e-mail addresses.
- Ignore damaged records.
- Continue even if a file is missing.

IMPLEMENTATION &DEMONTARATION:

1. validator.py

```python
def validate(parts):
    if len(parts) != 5:
        return False

    reg, name, dept, email, events = parts

    if not reg or not name or not dept or not events:
        return False

    if "@" not in email or "." not in email.split("@")[-1]:
        return False

    return True
```

2. reader.py

```python
import os
from .validator import validate

def read_files(folder):
    master = []
    seen = set()
    errors = []

    for i in range(1, 6):
        filename = f"dept{i}.txt"
        path = os.path.join(folder, filename)

        try:
            with open(path, "r", encoding="utf-8") as f:
                for line_no, line in enumerate(f, 1):
                    try:
                        parts = line.strip().split(",")

                        if not validate(parts):
                            raise ValueError("Invalid record")

                        reg = parts[0]

                        if reg in seen:
                            raise ValueError("Duplicate registration")

                        seen.add(reg)
                        master.append(parts)

                    except ValueError as e:
                        errors.append(
                            f"{filename}, Line {line_no}: {e}"
                        )

        except (FileNotFoundError, PermissionError, OSError) as e:
            errors.append(f"{filename}: {e}")

    return master, errors
```

3. report.py

```python
def generate_reports(master, errors):

    events = {}

    with open("master_list.txt", "w") as f:
        for record in master:
            f.write(",".join(record) + "\n")

            for event in record[4].split(";"):
                event = event.strip()

                if event:
                    events.setdefault(event, []).append(record[1])

    for event, names in events.items():
        with open(event + ".txt", "w") as f:
            for name in names:
                f.write(name + "\n")

    with open("summary.txt", "w") as f:
        f.write("Total Registrations: " + str(len(master)))

        for event, names in events.items():
            f.write(f"\n{event}: {len(names)}")

    with open("error_log.txt", "w") as f:
        for error in errors:
            f.write(error + "\n")
```

4.main.py

```python
from fest.reader import read_files
from fest.report import generate_reports

if __name__ == "__main__":

    master, errors = read_files("registrations")

    generate_reports(master, errors)

    print("Registration processing completed!")
    print("Valid Registrations:", len(master))
    print("Rejected Records:", len(errors))
```

Conclusion:

Using a set provides faster duplicate detection. Package organisation improves code reuse, readability, testing and maintenance. The program can process multiple department files without stopping because of invalid records or missing files.